MAHARANA INSTITUTE OF PROFESSIONAL STUDIES

Kanpur, Uttar Pradesh

SYNOPSIS FOR MINI PROJECT

on

TEXT SENTIMENT ANALYSER

(Sentiment and Emotion Analysis of English and Hinglish Text)

Course	:	B.Tech
Section	:	CSE-3B (MIPS)
Subject	:	Mini Project
Session	:	2026-2027

Submitted By:

Sudhanshu Shrivastav (2403490100035)

Audarsh Singh Rathore (2403490100015)

Ravi Kumar Bhardwaj (2403490100030)

Submitted To:

Mr. Abhishek Sir

Department of Computer Science and Engineering

Introduction

Sentiment analysis is a branch of Natural Language Processing (NLP) in which a computer reads a piece of text and finds out the opinion or feeling expressed in it, whether it is positive, negative or neutral. Today it is used by companies to understand product reviews, by brands to track their image on social media, and by researchers to study public opinion on different topics.

In India, the number of people using the internet and social media has grown very fast in the last few years. Most of these users do not write in pure English or pure Hindi. They mix both languages and type Hindi words using English letters, for example "yeh phone bahut mast hai but battery thodi weak hai". This style of writing is called Hinglish, and it is very common in YouTube comments, Instagram replies, WhatsApp messages and app reviews.

The problem is that most of the popular sentiment analysis tools are designed only for English. They do not understand Hindi words like *bakwas*, *zabardast* or *bekaar*, they cannot handle Hindi negation where *nahi* comes after the word, and they get confused because the same word is written in many spellings like *bahut*, *bahot* and *bhot*. Because of this, they give wrong results or mark most Hinglish comments as neutral, and a lot of useful customer feedback is lost.

Our project, **Text Sentiment Analyser**, is made to solve this problem. It is a web-based application that can analyse both English and Hinglish text and tell whether the text is positive, negative or neutral. It also detects emotions like joy, anger, sadness and fear present in the text. To do this, we use a combination of a rule-based method, which uses our own English and Hinglish word list with rules for negation and spelling variations, and machine learning models like Naive Bayes and Logistic Regression, which learn from labelled Hinglish comments. Both methods are combined in a hybrid model to get better accuracy.

The system also explains its result by showing which words affected the sentiment, can analyse many comments at once using a CSV file, and works fully inside the browser without sending the user's data to any server. In this synopsis, we explain the problem, our objectives, the related research work, the proposed system, its architecture and methodology, the requirements, the testing plan and the expected outcomes of the project.

1. Abstract

Most Indian users on YouTube, Instagram and shopping apps do not write in pure English or pure Hindi. They mix both and type Hindi in English letters, for example "movie acchi nahi thi but songs mast the". This mixed language is called Hinglish. Common sentiment analysis tools like VADER and TextBlob are made for English only, so they do not understand words like *bakwas*, *zabardast* or *bekaar*. They also get confused when the negation comes after the word (*acchi nahi*) and when the same word is spelled in many ways (*bahut*, *bahot*, *bhot*, *bohot*). Because of this, they mark most Hinglish comments as Neutral.

In this mini project we are building a system called **Text Sentiment Analyser** that reads English and Hinglish text and tells whether it is Positive, Neutral or Negative. It also shows the emotions present in the text using Plutchik's eight basic emotions. We have made a rule-based engine with a Hinglish word list, spelling correction and negation handling. We are also training two machine learning models, Naive Bayes and Logistic Regression, on Hinglish comments that we collect and label ourselves. Finally, we combine the rule-based output and the Naive Bayes output into one hybrid model.

We will compare five methods (English-only baseline, Hinglish rules, Naive Bayes, Logistic Regression and Hybrid) using k-fold cross-validation and report accuracy, precision, recall, F1 score and confusion matrices. The whole system runs inside the web browser, works without internet and does not send the user's text to any server.

Keywords: Sentiment Analysis, Hinglish, Code-Mixing, Emotion Detection, Naive Bayes, Logistic Regression, Hybrid Model

2. Problem Statement and Motivation

2.1 Problem Statement

To design and develop a system that can correctly find the sentiment (Positive, Neutral or Negative) and the main emotions in Hinglish and English text, where normal English-only tools fail because of Hindi words, different spellings of the same word and Hindi sentence structure.

2.2 Why Existing Tools Fail on Hinglish

While testing some popular tools on Hinglish comments, we noticed that they make the same kind of mistakes again and again. The table below shows the main reasons with examples.

Problem	Example	Output of English tool	Correct output
Hindi opinion words	"film ekdum bakwas thi"	Neutral (no known word)	Negative
Negation after the word	"movie acchi nahi thi"	Ignores *nahi*	Negative
Different spellings	"bhot badiya", "bohot badhiya"	Treats each spelling as unknown	Positive
Hindi intensifiers	"bahut zyada mast"	No extra weight given	Strongly Positive
Hindi contrast words	"camera mast hai lekin battery bekaar"	Does not know *lekin* means *but*	Negative
Stretched letters	"bahuuuut accha"	Unknown word	Positive
Sarcasm	"wah kya service hai, 2 ghante late"	Reads *wah* as praise	Negative

2.3 Motivation

A large part of Indian social media comments, product reviews and app reviews are written in Roman Hinglish. Companies like e-commerce sites, food delivery apps and film production houses want to know what their customers feel, but if a tool marks most of these comments as Neutral, the real complaints and the real praise both get missed.

We also noticed that most online sentiment tools only give a label and do not explain why the text was marked negative or positive. Cloud APIs also send the user's data to outside servers, which is not safe for private feedback, and big deep learning models need a GPU and a lot of labelled data. As students, we wanted to make something light that runs on any normal laptop, explains its answer, keeps the data private and works properly on Hinglish. This is the main motivation behind our project.

3. Objectives and Scope

3.1 Objectives

The main objectives of our project are:

1. To collect and label our own dataset of 600 to 2,000 Hinglish comments from YouTube, Instagram, X and Play Store, and check label quality using Cohen's kappa.

2. To build a rule-based engine that understands Hinglish words, spelling variations, English and Hindi negation, intensifiers, contrast words and emojis.
3. To train and compare Naive Bayes, Logistic Regression and Linear SVM models.
4. To combine the rule-based engine and Naive Bayes into a hybrid model.
5. To detect eight basic emotions in English and Hinglish text.
6. To evaluate all methods using k-fold cross-validation against an English-only baseline and develop an offline web application with a local REST API.

3.2 Scope

The project covers English and Hinglish text written in Roman (English) letters, from one sentence up to short paragraphs of around 5,000 characters. The system gives a three-class sentiment result, scores for eight emotions and the detected language (English, Hinglish or Roman Hindi). Users can analyse a single text, upload a CSV file for batch analysis or compare two texts. The system also shows which words affected the result, sentiment for different topics in the same comment (like camera or battery) and simple sarcasm hints.

Some things are not part of this project. Hindi written in Devanagari script is only detected and not analysed. Other code-mixed languages like Tamil-English or Bengali-English are not covered. Deep learning models like mBERT, IndicBERT and MuRIL are kept for future work. We are also not doing automatic scraping of social media; the data is collected manually.

4. Literature Survey

Before starting the project, we studied research papers on sentiment analysis and code-mixed text. A short summary is given below.

4.1 Rule-Based and Machine Learning Methods

Pang, Lee and Vaithyanathan (2002) used Naive Bayes, Maximum Entropy and SVM on movie reviews and showed that machine learning works better than simple keyword lists. Taboada et al. (2011) built SO-CAL, a lexicon-based method where word scores are changed by intensifiers and negation. Hutto and Gilbert (2014) proposed VADER, a rule-based model for social media that handles capital letters, exclamation marks, "but" and emoticons. Our scoring formula is based on VADER, but VADER only has English words and checks negation only before a word.

4.2 Code-Mixed and Indian Language Work

Bali et al. (2014) showed that English-Hindi code-mixing is very common on Facebook. Joshi et al. (2016) created a Hinglish sentiment dataset and used sub-word LSTM models to handle spelling variations. Patra, Das and Das (2018) and Patwa et al. (2020, SemEval SentiMix) gave large shared tasks on Hinglish tweets, where transformer models performed best. Khanuja et al. (2021) introduced MuRIL, which suits Hinglish but needs a GPU and a lot of data.

4.3 Emotion Models

Plutchik (1980) gave the theory of eight basic emotions, and Mohammad and Turney (2013) created the NRC Emotion Lexicon for English words. Neither of them covers Hinglish words.

4.4 Summary of Literature

Paper	Method	Language	Strength	Limitation
Pang et al. (2002)	NB, MaxEnt, SVM	English	ML better than keyword lists	No code-mixed text
Taboada et al. (2011)	Lexicon and rules	English	Works across domains	No Hindi words
Hutto and Gilbert (2014)	VADER	English	Good for social media	English words only
Bali et al. (2014)	Corpus study	Hinglish	Explains code-mixing	No classifier
Joshi et al. (2016)	Sub-word LSTM	Hinglish	Handles spellings	Needs GPU, not explainable
Patwa et al. (2020)	Transformers	Hinglish, Spanglish	Large benchmark	Heavy models
Khanuja et al. (2021)	MuRIL	Indian languages	Understands transliteration	Heavy, not explainable
Mohammad and Turney (2013)	Emotion lexicon	English	Eight emotions	No Hinglish words

4.5 Research Gap

From the above study, we found that there is no simple tool that does all of these together: understands Hinglish words and Hindi negation, learns from data, explains its result, detects emotions and runs offline on the user's own computer. Our project tries to fill this gap.

5. Existing System and Proposed System

5.1 Existing System

At present, if someone wants to find the sentiment of Hinglish comments, they mainly have three options:

1. English libraries like VADER and TextBlob. These are free and fast, but they give Neutral for most Hinglish text because they do not know Hindi words or Hindi negation.
2. Cloud NLP APIs from big companies. They support more languages, but every comment is sent to an outside server, they are paid, they need internet and they do not explain the result.
3. Research models like mBERT and MuRIL. These give high accuracy, but they need Python setup, a GPU, a big labelled dataset and good technical knowledge.

Limitations of the existing system:

- Low accuracy on Hinglish words and spelling variations
- No handling of Hindi word order like *accha nahi* or *pasand nahi aaya*
- Only a label is given, without any reason
- User data is sent to third-party servers
- High cost and hardware requirement for accurate models
- Mostly sentiment only, no emotion or topic-wise analysis

5.2 Proposed System

Our proposed system, Text Sentiment Analyser, is a hybrid sentiment analyser that understands Hinglish and runs completely inside the web browser. It combines a rule-based engine made by us with machine learning models trained on our own labelled data. The main features are:

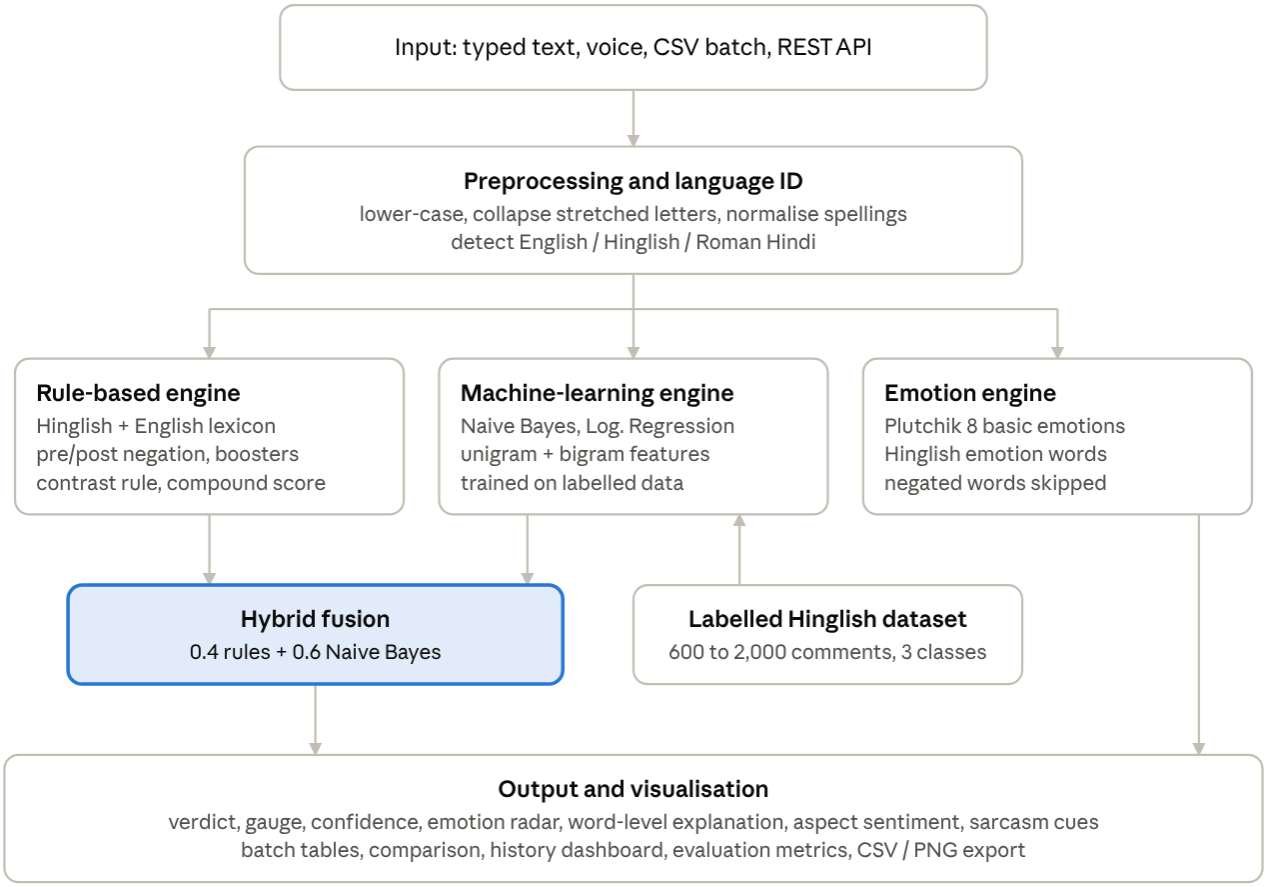
- Language detection: tells whether the text is English, Hinglish or Roman Hindi.
- Spelling normalisation: converts *bhot, bahot, bohot* to *bahut* and shortens stretched words.

- Negation in both directions: "not" affects the next words and *nahi* affects the previous words.
- Intensifiers (*bahut, ekdum,* very), dampeners (*thoda,* slightly) and contrast words (*but, lekin, magar*).
- Naive Bayes and Logistic Regression prediction with a hybrid of rules and Naive Bayes.
- Eight-emotion radar, word-level explanation ("Why this verdict?"), topic-wise sentiment and sarcasm hints.
- Full privacy: no outside libraries, history stays in the browser and the server runs only on 127.0.0.1.

6. System Architecture

The architecture of our system is shown in Fig. 1. The input text first goes through preprocessing and language detection. After that, three engines work on it at the same time. The outputs of the rule-based engine and the ML engine are combined in the hybrid step to give the final result, while the emotion engine sends its scores directly to the output.

System Architecture of Text Sentiment Analyser



System architecture of Text Sentiment Analyser

Fig. 1: System architecture of Text Sentiment Analyser

6.1 Description of Layers

1. **Input layer:** typed text, voice input, CSV file or REST API request.
2. **Preprocessing:** cleaning, spelling normalisation and language detection.
3. **Rule-based engine:** word scores with negation, intensifier, contrast and emoji rules.
4. **ML engine:** Naive Bayes and Logistic Regression class probabilities.
5. **Hybrid fusion:** weighted average of rule and Naive Bayes probabilities.
6. **Emotion engine:** scores for the eight emotions.
7. **Output layer:** result, charts, explanations and export, all drawn with SVG.

7. Methodology

Our work is divided into six steps: collecting and labelling data, preprocessing, rule-based scoring, machine learning, combining both (hybrid), and evaluation. Emotion detection runs side by side using the same cleaned words.

7.1 Data Collection and Labelling

We are collecting public comments from YouTube, Instagram, X and Play Store reviews (target 600 to 2,000 comments, almost equal in each class). The data is saved in a CSV file with original spellings and emojis, without any usernames. A comment is labelled Positive if it shows a good opinion, Negative if it complains or shows anger or sadness, and Neutral if it is a fact or question. Two of us label the same 100 comments separately, and we calculate Cohen's kappa:

$$\kappa = (p_o - p_e) / (1 - p_e)$$

Here p_o is the observed agreement and p_e is the agreement expected by chance. A value above 0.6 means good agreement.

7.2 Preprocessing

1. Convert the text to lower case.
2. Reduce letters repeated three or more times to two (*bahuuuut* becomes *bahuut*).
3. Split the text into words, emojis, emoticons and punctuation marks like ! and ?.
4. Replace different spellings with one standard form using a dictionary (*bhot, bahot, bohot, bht* become *bahut*; *acha, achha, acchi* become *accha*; *nhi, nai, nahin* become *nahi*; *osm, awsm* become *awesome*).

5. Detect the language by counting how many words are from our Roman Hindi word list. Words like *me, to, main, so* and *par* are not counted because they are also English words.

7.3 Rule-Based Sentiment Engine

Every word in our word list has a score between -4 and +4. The English list has around 290 words and the Hinglish list has around 180 words (for example *zabardast* +4, *mast* +3, *bakwas* -4, *bekaar* -3). These scores are then changed using the rules given below.

Rule	Example words	What it does
Intensifier	very, extremely, bahut, ekdum, zyada	Score is multiplied by 1.6
Dampener	slightly, thoda	Score is multiplied by 0.6
English negation	not, never, don't, no	Next 3 words are multiplied by -0.75
Hindi negation	nahi, nahin, nhi	Previous 3 words are multiplied by -0.75
Contrast	but, lekin, magar	Part before is halved, part after is multiplied by 1.5
Emphasis	!, words in CAPITALS	Score is increased slightly
Emojis and emoticons	😍 🔥 😣 👎 :) :(	Have their own score and are not flipped by negation

All the changed scores are added to get a total x, which is converted into a compound score between -1 and +1:

$c = x / \sqrt{(x^2 + 15)}$

If c is 0.05 or more the text is Positive, if c is -0.05 or less it is Negative, and otherwise it is Neutral. When we switch off the Hinglish word list and Hindi negation, we get the English-only baseline that we use for comparison.

7.4 Machine Learning Models

For machine learning, we use single words (unigrams) and pairs of words (bigrams) as features, so that phrases like *accha nahi* and *paisa vasool* are also learned.

Multinomial Naive Bayes uses Laplace smoothing, where the probability of a feature f in class y is:

$P(f \mid y) = (\text{count}(f, y) + \alpha) / (\text{total words in class } y + \alpha \times |V|)$

where α = 1 and |V| is the number of unique features.

Logistic Regression (softmax) is trained on TF-IDF features using stochastic gradient descent with L2 regularisation. In Python, we also train a Linear SVM (C = 1.0) for comparison.

7.5 Hybrid Model

The compound score c from the rule engine is changed into probabilities: P(Positive) = max(c, 0), P(Negative) = max(-c, 0), and the rest goes to Neutral. These are then combined with the Naive Bayes probabilities:

P_hybrid(y) = 0.4 × P_rules(y) + 0.6 × P_NB(y)

The class with the highest P_hybrid is the final answer. The rules help with negation and contrast, which a small dataset cannot teach properly, and Naive Bayes helps with domain words that are not in our word list. The weights 0.4 and 0.6 will be tuned during testing.

7.6 Emotion Detection

We use an emotion word list in which English and Hinglish words and emojis are linked to one or more of the eight emotions. For example, *khush* and *maza* are joy, *gussa* and *dhokha* are anger, *darr* and *tension* are fear, and *udaas* and *dukh* are sadness. Words that are negated are skipped, so "gussa nahi hai" is not counted as anger. The scores are shown on a radar chart, and the highest one is shown as the main emotion.

7.7 Evaluation

We will use stratified 5-fold cross-validation with a fixed random seed (42), so each fold has the same class balance. For every method we will calculate accuracy, macro precision, macro recall and macro F1 score (average and standard deviation over the folds), along with confusion matrices. We will also study the wrongly classified comments and group them into sarcasm, unknown spellings, mixed opinions and domain words.

The final results will be reported only on our own collected dataset. The small sample dataset of 210 comments is used only for testing the code, because the Hinglish word list was made along with it and its scores would be too high.

8. Implementation and Requirements

8.1 Modules of the Application

S. No.	Module	Description
1	Analyzer	Analyses one text and shows result, emotions, important words and topic-wise sentiment
2	Batch	Analyses a CSV file of many comments and exports the results
3	Compare	Compares two texts side by side
4	Dashboard	Shows history, mood trend and activity
5	Evaluation	Tests all five methods on a labelled CSV and shows metrics and confusion matrices

8.2 Hardware Requirements

Component	Minimum Requirement
Processor	Any dual-core processor
RAM	4 GB
Storage	100 MB free space
Display	1366 x 768 or above (also works on mobile)
Internet	Not needed to run, only for collecting data

8.3 Software Requirements

Software	Version	Use
Operating System	Windows 10/11, Linux or macOS	Development and running
Web Browser	Latest Chrome, Edge or Firefox	Running the application
Node.js	18 or above	Local server, API and testing
Python	3.10 or above	Experiments for report
Python Libraries	scikit-learn, pandas, NumPy, Matplotlib	Models, results and graphs

Software	Version	Use
Code Editor	Visual Studio Code	Writing code
Version Control	Git	Managing code

9. Testing Plan and Expected Outcomes

9.1 Testing Plan

We will do unit testing of every engine using Node.js assert, integration testing of the REST API, model testing using 5-fold cross-validation, and user testing with 5 to 10 classmates on Chrome, Edge and Firefox.

9.2 Sample Test Cases

Test ID	Test Case	Input	Expected Output
T1	English positive	"The delivery was fast and the food was delicious"	Positive
T2	Hindi negation	"movie acchi nahi thi"	Negative
T3	Spelling variation	"bhot badiya"	Positive
T4	Contrast word	"camera mast hai but battery bekaar"	Negative
T5	Neutral sentence	"match kal 7 baje hai"	Neutral
T6	Language detection	"movie bahut acchi thi yaar"	Hinglish
T7	Negated emotion	"gussa nahi hai"	Anger score = 0
T8	API wrong input	Wrong JSON	HTTP 400

9.3 Expected Outcomes

1. A labelled Hinglish dataset of 600 to 2,000 comments with sources, labelling rules and agreement score, which can also be used by other students.
2. Clear improvement of the Hinglish-aware methods over the English-only baseline, which we expect to mark most Hinglish comments as Neutral.

3. A comparison showing whether the hybrid model performs better than the rule engine and Naive Bayes alone.
4. A list of the cases that are still difficult, like sarcasm, new spellings and mixed opinions, which will guide future work.
5. A working web application and REST API that runs offline and keeps the user's data private.

10. Project Plan, Applications and Conclusion

10.1 Project Timeline

Phase	Work to be Done	Duration (Weeks)
1. Study and planning	Literature survey, problem definition, requirements, synopsis	1 to 3
2. Data collection	Collecting comments, labelling rules, labelling, agreement check	3 to 7
3. Core engines	Language detection, normalisation, rule engine, emotion engine	5 to 9
4. ML and hybrid	Naive Bayes, Logistic Regression, hybrid model, cross-validation, Python experiments	8 to 11
5. Application	Analyzer, Batch, Compare, Dashboard and Evaluation modules, REST API	9 to 13
6. Testing	Unit, integration and user testing, final results, error analysis	12 to 14
7. Documentation	Project report, presentation and demo	14 to 16

10.2 Applications

- E-commerce and food delivery: finding what customers like or dislike in reviews.
- Social media monitoring: tracking reactions to brand posts, movies and trailers.
- Customer support: finding angry or sad messages so they are handled first.
- Education: showing how negation and contrast words change sentiment.

10.3 Limitations

- Sarcasm is detected only through simple hints, so some sarcastic comments may still be wrong.
- Words that are neither in our word list nor in the training data cannot be scored properly.
- Hindi in Devanagari script and other Indian languages are not supported.
- Emotion detection based on a word list may not catch mixed or hidden feelings.
- The accuracy depends on the size and quality of the dataset we collect.

10.4 Future Scope

- Fine-tuning mBERT, IndicBERT and MuRIL on our dataset and comparing with our hybrid model.
- Adding a separate sarcasm detection model and learning the hybrid weights automatically.
- Supporting Devanagari script and other code-mixed languages.
- Making a browser extension or mobile app for live comment analysis.

10.5 Conclusion

In India, a large number of people write online in Hinglish, but most sentiment analysis tools only understand English. Through this project, we are trying to solve this problem by combining a Hinglish word list with spelling correction and Hindi negation rules, simple machine learning models and a hybrid model. Our system will not only give the sentiment of the text but also explain why it gave that result. By testing on our own labelled dataset and comparing with an English-only baseline, we will be able to show how much each part improves the results. Since the whole system runs in the browser, it is free, works offline and keeps the user's data private.

References

1. B. Pang, L. Lee and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *Proc. EMNLP*, 2002, pp. 79-86.
2. M. Taboada, J. Brooke, M. Tofiloski, K. Voll and M. Stede, "Lexicon-based methods for sentiment analysis," *Computational Linguistics*, vol. 37, no. 2, pp. 267-307, 2011.
3. C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. ICWSM*, 2014.

4. K. Bali, J. Sharma, M. Choudhury and Y. Vyas, "'I am borrowing ya mixing?' An analysis of English-Hindi code mixing in Facebook," in *Proc. Workshop on Computational Approaches to Code Switching*, 2014.
5. A. Joshi, A. Prabhu, M. Shrivastava and V. Varma, "Towards sub-word level compositions for sentiment analysis of Hindi-English code mixed text," in *Proc. COLING*, 2016.
6. B. G. Patra, D. Das and A. Das, "Sentiment analysis of code-mixed Indian languages: An overview of SAIL_Code-Mixed shared task @ICON-2017," arXiv:1803.06745, 2018.
7. P. Patwa et al., "SemEval-2020 Task 9: Overview of sentiment analysis of code-mixed tweets," in *Proc. SemEval*, 2020.
8. S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730, 2021.
9. R. Plutchik, "A general psychoevolutionary theory of emotion," in *Emotion: Theory, Research, and Experience*, vol. 1, Academic Press, 1980.
10. S. M. Mohammad and P. D. Turney, "Crowdsourcing a word-emotion association lexicon," *Computational Intelligence*, vol. 29, no. 3, pp. 436-465, 2013.